

2026-10-01

equivR: Reproducible Cross-Runtime Equivalence for R Analytics

Yonglin Zhu 

Independent open-source project
zhuygln@gmail.com

Xing Cai 

Independent open-source project
caixingt@gmail.com

Executive Summary

equivR is an open-source cross-runtime validation toolkit for R modernization, migration, and refactoring. R is the community focus; the implementation language of a trusted reference or candidate is outside the trust model, and Python may be used internally for orchestration.

Generation proposes; validation decides.

A translator, LLM, or developer may produce code, but acceptance requires execution against an explicit equivalence contract. Any failed required check blocks acceptance and returns deterministic diagnostics for repair and re-validation.

A minimal proof of concept already demonstrates exact, tolerance-aware, and structural checks. The project also builds on prior delivery of a completed SAS-to-Python modernization framework using AI-assisted transformation, hybrid execution, and independent numerical validation. The grant turns these proven patterns into reusable R-centered infrastructure: a language-neutral contract, cross-runtime artifact normalization, structured-result validation, reproducibility evidence, R-facing workflows, and community benchmark cases.

Signatories

Project team

Yonglin Zhu - co-PI / co-lead; Senior Staff Scientist at SAS Institute. ORCID [0000-0003-0245-827X](https://orcid.org/0000-0003-0245-827X); zhuygln@gmail.com. Led development of a completed SAS-to-Python modernization framework using

AI-assisted transformation, hybrid SAS/Python execution, and numerical-equivalence validation. Leads architecture, equivalence-contract design, and validation workflow implementation. Project: <https://github.com/zhuygl/equivR>.

We request **\$5,000 over four months** for project labor. No funds are requested for cloud services or AI credits.

Xing Cai - co-PI / co-lead; founding member of the R Working Group at PPD. ORCID [0009-0004-5983-3338](https://orcid.org/0009-0004-5983-3338); caixingt@gmail.com. Applied analytics experience with SAS, R, Python, statistical modeling, and data-science workflows; co-leads R-community use-case design, benchmark development, testing, documentation, and applied evaluation.

equivR is independent open-source work: it is not sponsored, funded, or endorsed by either co-PI's employer and does not depend on employer-owned code, data, or infrastructure.

Contributors / consulted. None at submission time; community contributions will be invited through the public repository and benchmark process. The design is informed by public R Consortium Submissions Working Group materials ([R Consortium 2026c](#), [2026b](#), [2026a](#)); citation does not imply endorsement.

The Problem

Modernizing analytical software - legacy R to modern R, SAS or Python to R, package replacement, refactoring, or AI-assisted rewrite - raises a question beyond whether the new code runs: does it preserve the analytical behavior that matters? This affects R developers, maintainers, and analytical teams that need independent evidence before adopting a new implementation.

Today acceptance criteria such as keys, missing-value rules, ignored metadata, numerical tolerances, estimates, and confidence intervals are often scattered across project-specific tests. AI increases the risk: plausible code can be subtly wrong, and self-review by the same model is not independent evidence.

The R Consortium Submissions Working Group demonstrates the value of reference-based validation: Pilot 5 compares R-generated ADaM datasets with prior reference datasets in CI ([R Consortium 2026c](#)), while Working Group materials emphasize reproducibility, reviewability, and expanding AI/automation work ([R Consortium 2026a](#), [2026b](#)). Existing object/data-frame comparison tools cover pieces of this workflow, but not a reusable layer joining declared contracts, cross-runtime normalization, analytical results, provenance, CI gating, diagnostics, and re-validation. **equivR** targets that gap for the R community.

The proposal

Overview

Over four months, **equivR** will provide cross-runtime validation for R modernization. Trusted references may originate in legacy R, SAS, Python, or another runtime; candidates may be modern R, refactors, package migrations, or AI-assisted code. Runtime adapters normalize artifacts, a deterministic core applies a language-neutral equivalence contract, and R-facing CLI/wrapper/CI workflows expose results.

Detail

Minimum Viable Product

The Phase 1 MVP will let a user:

1. provide trusted reference artifacts and a candidate in an R modernization workflow;
2. declare keys, schema/missingness rules, ignored fields, numerical tolerances, and required analytical outputs;
3. normalize and compare datasets and selected structured results;
4. receive deterministic pass/fail status, repair-oriented diagnostics, reproducibility metadata, and machine-/human-readable evidence; and
5. invoke validation from an R-friendly CLI or thin wrapper and CI.

Version 1 covers row identity, required variables, missing values, exact categorical values, tolerance-aware numeric values, and named results such as estimates, confidence intervals, counts, and p-values.

The initial benchmark suite will include **(1) legacy R -> modern R** and **(2) cross-runtime trusted reference -> R**, using pregenerated artifacts so no licensed SAS runtime is required in CI. A documented community format will support additional package-replacement, refactoring, Python-to-R, SAS-to-R, or AI-assisted R cases.

Scope boundary. This grant does not fund a general translator, coding agent, live SAS integration, regulatory certification, or universal program-equivalence proof. The deliverable is the independent validation infrastructure and R-centered benchmark workflows.

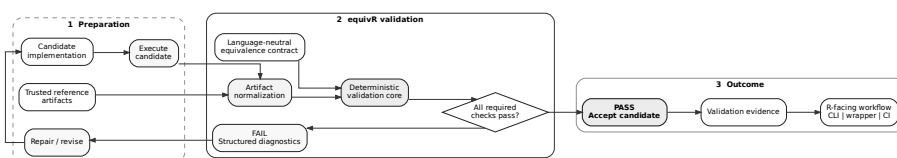


Figure 1: equivR validation workflow. Candidate implementations execute and emit artifacts that are normalized alongside trusted reference artifacts; the deterministic core applies a language-neutral equivalence contract and requires all declared checks to pass before acceptance, otherwise returning structured diagnostics that drive repair and re-validation. R-facing CLI, wrapper, and CI entry points consume validation evidence.

Architecture

equivR separates: (1) a language-neutral contract; (2) runtime adapters/artifact normalization; (3) a deterministic validation core implemented in R and/or Python; (4) an R-facing workflow layer; and (5) evidence generation. A bounded LLM-revised R example demonstrates **generate -> validate -> diagnose -> repair -> revalidate**.

LLM dependency and model independence. The closed-loop equivR workflow depends on an LLM to generate or revise candidate code in response to validation diagnostics. However, the framework is intentionally **model-provider independent**: it does not rely on a particular proprietary API, model family, or vendor. Any sufficiently capable contemporary general-purpose LLM can be used through the model interface, including leading proprietary or open-weight models available in 2026. The LLM proposes transformations or repairs; deterministic equivR validation remains the authority that decides whether a candidate is accepted.

Assumptions

Users can identify a trusted reference and express important criteria through structured datasets and selected analytical results, using explicit tolerances where appropriate. Delivery risk is controlled by limiting Phase 1 to artifact-based validation, pregenerated cross-runtime references, small adapters, two benchmark cases, and a community contribution format. Behavior outside the declared contract is not claimed to be verified.

External dependencies

The project uses open-source R/Python libraries and standard CI tooling, with no licensed-SAS dependency; proprietary reference artifacts can be generated beforehand. **equivR** requires access to an LLM for its automated generate-validate-diagnose-repair workflow, but not to any specific model or provider. The implementation will expose a model-agnostic interface so users can select among capable contemporary LLMs. Open/local models and proprietary hosted APIs are both supported. Proprietary API charges used during project evaluation will be self-funded or provided in-kind rather than charged to the ISC grant.

Project plan

Start-up phase

Month 1. Finalize governance, **MIT license**, contribution guidance, CI/reporting, the contract schema, validation-result model, and core/adaptor boundary. The existing proof of concept already demonstrates exact-match, within-tolerance, mismatch, and structural-failure cases.

Technical delivery

M1 - Contract and dataset validation (Month 1-2) - \$1,250. Implement keyed matching, schema/missingness checks, exact categorical and tolerance-aware numeric comparison, and structured diagnostics; public fixtures show pass/fail behavior.

M2 - Cross-runtime normalization and structured results (Month 2) - \$1,000. Add artifact adapters and comparison of named results; one example validates pregenerated cross-runtime artifacts against an R candidate without a proprietary runtime in CI.

M3 - R-facing workflow and benchmark suite (Month 2-3) - \$1,000. Provide an R-friendly CLI/thin wrapper and reference CI workflow; both initial benchmark cases run from a clean checkout with a documented contribution path.

M4 - Evidence and closed-loop validation (Month 3-4) - \$750. Capture R/Python session metadata, source revision, checksums, and machine-readable results; demonstrate a failed candidate repaired and accepted only after all required contract checks pass.

M5 - Documentation and release (Month 4) - \$1,000. Complete user/architecture/contribution documentation, benchmark examples, and release hardening; publish an initial tagged release.

Other aspects

Development is public at <https://github.com/zhuygl/equivR> under the **MIT License**, with a documented community benchmark format and feedback through GitHub issues/discussions. We will publish project announcement and completion/update material suitable for R Consortium/community channels.

Budget & funding plan

Milestone	Amount
Contract and dataset validation	\$1,250
Cross-runtime normalization and structured results	\$1,000
R-facing workflow and benchmark suite	\$1,000
Evidence and closed-loop validation	\$750
Documentation and release	\$1,000
Total	\$5,000

No grant funds are requested for travel, workshops, hardware, publication fees, cloud services, AI credits, indirect costs, or licensed SAS software.

Success

Definition of done

An R user can provide reference artifacts and a candidate, declare a contract, run validation locally or in CI, and receive deterministic diagnostics and reproducible evidence without the reference runtime present. Both initial benchmark cases run from a clean checkout, and one closed-loop example accepts a repaired candidate only after **100% of required declared checks pass**. A contribution specification supports additional community cases.

Measuring success

Success means: CI passes; both benchmark cases reproduce from a clean checkout; deterministic machine- and human-readable evidence is produced; the closed-loop example passes; an R-friendly invocation path and benchmark contribution specification are documented; and an initial tagged release is published. Any declared mismatch must fail closed with actionable diagnostics.

Future work

Future consumers may include live SAS/other runtime adapters, richer result schemas, hybrid SAS/R or Python/R orchestration, repository-scale translation, and automated multi-step repair - not Phase 1 deliverables.

R Consortium. 2026a. “Making r Submissions Reviewable for FDA.” <https://r-consortium.org/posts/making-r-submissions-reviewable-for-fda/>.

R Consortium. 2026b. “R Submissions Working Group: 2026 Plans.” <https://r-consortium.org/posts/submissions-wg-2026/>.

R Consortium. 2026c. “Submissions Pilot 5 Dataset-JSON Repository.” <https://github.com/RConsortium/submissions-pilot5-datasetjson>.